Cronograma del Proyecto Integrador

SaaS Multitenant · Next.js + TypeORM + Supabase PostgreSQL

Objetivo: desarrollar una plataforma SaaS para inmobiliarias, con aislamiento por schema de PostgreSQL por tenant. Se prioriza el desarrollo funcional y se dejan Docker y CI/CD para el final.

Día 1 — Supabase + TypeORM

Tiempo estimado: 2–4 h **Prioridad:** Prioridad máxima

• Crear/configurar el proyecto de Supabase y Next.js.

• Conectar TypeORM a PostgreSQL.

• Probar una entidad global en `public`.

• Crear un schema de prueba y consultar una tabla dentro de ese schema.

Día 2 — Registro, login y sesión

Tiempo estimado: 2–4 h

• Implementar hash seguro de contraseñas.

• Crear autenticación con JWT en cookie `httpOnly`.

• Proteger rutas y obtener el tenant desde la sesión validada.

Día 3 — Multitenancy real

Tiempo estimado: 2–4 h

• Implementar el alta de un tenant.

• Crear automáticamente su schema y las tablas de negocio.

• Probar el aislamiento entre dos inmobiliarias.

Día 4 — CRUD de inmobiliaria

Tiempo estimado: 3–4 h

• Crear, consultar, actualizar y eliminar propiedades, interesados (leads) y visitas.

• Validar los datos del backend con Zod.

Día 5 — Idempotencia y rate limiting

Tiempo estimado: 3–4 h

• Implementar dos escrituras idempotentes y prevenir duplicados.

• Aplicar límites de solicitudes a login y endpoints de creación.

• Responder con HTTP 429 y la cabecera `Retry-After` al exceder los límites.

Día 6 — Pruebas, interfaz y documentación

Tiempo estimado: 2–4 h

• Verificar aislamiento, seguridad e idempotencia.

• Revisar migraciones y estrategias de renderizado.

• Documentar la arquitectura y el funcionamiento en el README.

Día 7 — Docker y CI/CD

Tiempo estimado: 2–4 h

• Crear `Dockerfile` y `docker-compose.yml`.

• Preparar `.env.example` sin secretos reales.

• Configurar GitHub Actions para instalar dependencias y ejecutar lint, build o pruebas.

• Ensayar la sustentación del proyecto.

Arquitectura acordada

Área	Decisión
Aplicación	Next.js App Router + TypeScript; monolito modular organizado por funcionalidades.
Base de datos	Supabase PostgreSQL; datos globales en public y un schema por tenant.
ORM	TypeORM; gestionar cambios mediante migraciones y evitar synchronize automático en producción.
Features	properties, leads y visits; cada módulo puede organizar components, services, types, utils y schemas.
Shared	Componentes, tipos, utilidades y constantes genéricas realmente compartidas.
Infraestructura	lib/db, lib/auth, lib/tenancy, lib/idempotency y lib/rate-limit.
Seguridad	JWT en cookie httpOnly, hash de contraseñas, validación backend y schema resuelto en el servidor.

Prioridades y alcance

• Completar primero los requisitos obligatorios; no añadir IA, pagos, mapas o realtime antes del núcleo funcional.

• La prueba del día 1 debe confirmar que TypeORM puede crear schemas y consultar tablas con la conexión elegida de Supabase.

• No confiar en nombres de schema enviados por el navegador; derivarlos de datos internos validados.

• No guardar secretos reales en el repositorio. Usar variables de entorno locales y un .env.example con valores de muestra.